

RAPPORT TECHNIQUE

Plateforme Koobo

Intelligence Artificielle au Service de l'Agriculture

Module 1 : Chatbot RAG Agricole
Module 2 : Detection de Maladies par Vision IA
Module 3 : Recommandation de Cultures ML

Version 1.0 | Fevrier 2025

1. Introduction et Contexte

L'agriculture moderne fait face a des defis croissants : variabilite climatique, pression des ravageurs, appauvrissement des sols et difficulte d'acces a l'expertise agronomique pour les petits exploitants. Dans ce contexte, l'intelligence artificielle offre des opportunités inédites pour démocratiser le conseil agricole.

La plateforme Koobo répond a cette problematique en proposant un outil numerique integre, combinant trois technologies d'IA complementaires : la generation augmentee par la recherche (RAG) pour le conseil, la vision par ordinateur pour le diagnostic phytosanitaire, et l'apprentissage automatique supervise pour l'aide a la decision culturale. Chaque module adresse un besoin specifique et concret des agriculteurs.

Objectif	Fournir aux agriculteurs un assistant IA accessible, precis et multimodal pour optimiser leurs decisions agronomiques.
----------	--

2. Architecture Generale du Systeme

L'application est construite sur une architecture trois tiers classique, avec une separation claire entre le frontend HTML/CSS/JS, le backend Flask Python et les services d'IA externes. Deux instances Flask coexistent, isolant le module de recommandation de cultures (port 5001) du coeur applicatif (port 5000).

2.1 Composants Principaux

Le backend repose sur Flask, un micro-framework Python reconnu pour sa legerte et sa flexibilite. La gestion de l'etat conversationnel est assuree par les sessions Flask cote serveur, serialisant et deserialisant les messages LangChain a chaque requete. L'orchestration des chaines LLM est confiee a LangChain, qui abstrait les interactions avec les differents modeles de langage.

Composant	Technologie	Role
Backend Web	Flask (Python)	Serveur HTTP, gestion des routes et sessions
LLM - Chatbot	LLaMA 3 70B / Groq	Generation de reponses conversationnelles
LLM - Vision	Gemini Pro Vision	Analyse d'images et diagnostic maladies
Embeddings	Google Gen AI Embed.	Vectorisation des documents agricoles
Vector Store	ChromaDB	Stockage et recherche semantique

Composant	Technologie	Role
Orchestration	LangChain	Chaines RAG et retrieval contextuel
ML Cultures	Random Forest (sklearn)	Classification de cultures adaptees
Meteo	OpenWeatherMap API	Donnees temperature / humidite en temps reel

3. Description des Modules Techniques

3.1 Module Chatbot RAG (Retrieval-Augmented Generation)

Ce module constitue le coeur informationnel de la plateforme. Il met en oeuvre un pipeline RAG complet via LangChain, permettant au systeme de repondre a des questions agricoles en s'appuyant sur une base documentaire vectorisee plutot que sur les seules connaissances parametriques du LLM.

Le flux de traitement se deroule en plusieurs etapes successives. A la reception d'une requete utilisateur, l'historique conversationnel est deserialisee depuis la session Flask. Un retriever histoire-conscient (`history_aware_retriever`) reformule la question en tenant compte du contexte anterieur. Cette question reformulee est utilisee pour rechercher les $k=2$ chunks les plus pertinents dans ChromaDB via similarite cosinus. Enfin, le LLaMA 3 70B synthetise une reponse en combinant le contexte recupere et l'historique.

Le vecteur store ChromaDB est pre-indexe avec les embeddings Google (models/embedding-001), garantissant une coherence entre l'indexation et la recherche. La persistance du repertoire stores/koobo_cosine/ permet un chargement rapide sans re-indexation.

3.2 Module de Detection de Maladies (Vision IA)

Ce module exploite les capacites multimodales de Gemini Pro Vision pour analyser des images de plantes soumises par les agriculteurs. L'image est d'abord convertie en base64 cote serveur via la bibliotheque PIL, puis transmise au modele via le format `image_url` supporte par LangChain Google.

L'analyse s'effectue en quatre appels successifs au modele, chacun focalise sur un aspect specifique : identification de la maladie, determination de ses causes, description des symptomes visibles, et recommandation de traitements. Ce design multi-passes permet d'obtenir des reponses plus detaillees et coherentes qu'un prompt unique, le nom de la maladie identifie lors du premier appel etant injecte dans les suivants pour maintenir la coherence thematique.

Un second endpoint (`/get_detect_chat`) offre une interaction libre avec l'image, permettant a l'agriculteur de poser des questions specifiques sur la plante analysee.

3.3 Module de Recommandation de Cultures (Machine Learning)

Ce module s'appuie sur un modele Random Forest pre-entraine, serialise dans le fichier `rf_model.pkl`. Le modele prend en entree un vecteur de 7 variables agronomiques et climatiques pour predire la culture la mieux adaptee parmi 22 categories.

Variable d'entree	Type	Source
Azote (N)	Numerique (int)	Saisie utilisateur
Phosphore (P)	Numerique (int)	Saisie utilisateur
Potassium (K)	Numerique (int)	Saisie utilisateur
pH du sol	Numerique (float)	Saisie utilisateur
Pluviometrie	Numerique (float)	Saisie utilisateur
Temperature	Numerique (float)	API OpenWeatherMap
Humidite	Numerique (float)	API OpenWeatherMap

La prediction est automatiquement traduite en francais via un dictionnaire de correspondances couvrant l'ensemble des 22 cultures supportees. La recuperation des donnees meteorologiques depuis l'API OpenWeatherMap est assuree par la fonction `utils.weather_fetch()`, qui prend en parametre le nom de la ville saisie par l'utilisateur.

4. Securite et Bonnes Pratiques

La gestion des secrets sensibles (cles API, cle secrete Flask) est assuree via le module `python-dotenv`, chargeant les variables depuis un fichier `.env` exclu du versionnement. La cle secrete Flask est utilisee pour signer les cookies de session, protegeant l'integrite de l'historique conversationnel.

Les images soumises par les utilisateurs sont traitees entierement en memoire (flux `BytesIO`), sans ecriture sur disque, limitant les risques lies au stockage de contenus non valides. Le `LANGCHAIN_TRACING_V2` est active pour un monitoring complet des chaines LLM via `LangSmith`, facilitant le debogage en production.

Aspect Securite	Mesure implementee
Gestion des secrets	Variables d'environnement via <code>.env</code> / <code>python-dotenv</code>
Sessions	Signature cryptographique via <code>FLASK_SECRET_KEY</code>
Upload d'images	Traitement en memoire, pas de stockage disque
Monitoring LLM	LangChain Tracing V2 (<code>LangSmith</code>)
Separation des services	Isolation sur ports distincts (5000 / 5001)

5. Analyse Critique et Perspectives

5.1 Points Forts

- Architecture modulaire permettant une evolution independante de chaque composant IA
- Approche multi-LLM optimisant cout et performance selon la nature de la tache
- Integration transparente de donnees temps reel (meteo) dans le pipeline ML
- Experience utilisateur enrichie par la memoire conversationnelle persistante

5.2 Axes d'Amelioration Identifies

- Consolidation en une seule application Flask pour simplifier le deploiement
- Mise en cache de la connexion ChromaDB (actuellement rechargee a chaque requete)
- Ajout d'une validation et sanitisation des entrees utilisateur (formulaires, uploads)
- Migration vers gemini-1.5-pro-vision, version plus recente et performante
- Mise en place de tests unitaires et d'integration pour les chaines LangChain

5.3 Evolutions Futures Recommandees

A court terme, la mise en place d'un systeme de cache pour les embeddings et la connexion ChromaDB reduirait significativement la latence des requetes chatbot. A moyen terme, l'integration d'un module de detection d'insectes (actuellement commente dans le code) elargirait considerablement la valeur de la plateforme. A plus long terme, le deploiement d'un modele de vision fine-tune sur des donnees agricoles specifiques a la region cible ameliorerait la precision du diagnostic.

Conclusion

La plateforme Koobo illustre avec succes la complementarite des differentes approches d'IA pour repondre a des besoins agricoles concrets. En combinant RAG, vision multimodale et apprentissage automatique supervise dans une interface web unifiee, elle propose une solution a forte valeur ajoutee pour les agriculteurs. Les axes d'amelioration identifies — principalement sur la performance et la robustesse — constituent une feuille de route claire pour les prochaines iterations du produit.

Stack

Python · Flask · LangChain · LLaMA 3 · Gemini Vision · ChromaDB ·
Random Forest · OpenWeatherMap